



Tuyển tập Bài tập Thực hành: Kỹ thuật Mô phỏng & Hiện thực hóa (Simulation & Implementation)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)
Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool
Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Danh mục Bài tập Phân cấp (Problem List)

STT	Tên bài toán	Nền tảng & ID	Cấp độ	Kỹ thuật cốt lõi
1	Weird Algorithm	CSES 1068	● Cơ bản	Mô phỏng giả thuyết Collatz $3n+1$, tránh tràn long long
2	Repetitions	CSES 1069	● Cơ bản	Duyệt chuỗi một lượt, duy trì độ dài khối ký tự liên tiếp
3	Spiral Numbers	CSES 1071	● Cơ bản	Mô phỏng đường đi xoắn ốc trên lưới vô hạn, rút gọn $O(1)$
4	Takahashi's Information	AtCoder ABC 088 C	● Cơ bản	Kiểm tra tính nhất quán của bảng ma trận 3×3
5	Snuke's Coloring	AtCoder ABC 045 C	● Cơ bản	Mô phỏng bảng điểm thừa bằng bảng băm / map
6	Boring Apartments	Codeforces 1433A	● Cơ bản	Mô phỏng quá trình gỡ phím tuần tự theo chữ số lặp
7	Conway's Game of Life	LeetCode 289	● Cơ bản	Lưới 2D, cập nhật đồng thời bằng Double Buffering
8	Grid Repainting 2	AtCoder ABC 096 C	● Cơ bản	Mảng hướng di chuyển 4 phía, kiểm tra tính hợp lệ lân cận
9	Takahashi and Meatballs	AtCoder ABC 175 B	● Cơ bản	Duyệt bộ ba, kiểm tra điều kiện hình học tam giác
10	Walking Robot	Codeforces 1154D	● Trung cấp	Mô phỏng có tham lam năng lượng pin và sạc quang điện

STT	Tên bài toán	Nền tảng & ID	Cấp độ	Kỹ thuật cốt lõi
11	Takahashi on a Grid	AtCoder ABC 275 C	● Trung cấp	Mô phỏng 4 đỉnh hình vuông trên ma trận 9×9
12	Takahashi's Tour	AtCoder ABC 213 D	● Trung cấp	Mô phỏng hành trình DFS tuần tự có thứ tự từ điển
13	Snuke's Cookie / Teleportation	AtCoder ABC 167 D	● Trung cấp	Mô phỏng $K \leq 10^{18}$ bước chuyển bằng Cycle Finding
14	Robot Vacuum Cleaner	Codeforces 922D	● Trung cấp	Mô phỏng ghép xâu và tham lam so sánh hoán vị
15	Bank Queue Simulation	CSES 1164 (Room Allocation)	● Trung cấp	Mô phỏng hướng sự kiện bằng Priority Queue
16	Snake Game Simulation	LeetCode 353	● Trung cấp	Mô phỏng game Rắn săn mồi bằng Deque + Set tọa độ
17	2048 Game Engine	Classical Game Engine	● Nâng cao	Mô phỏng trượt hàng/cột, dồn ô và gộp số trong game 2048
18	The Game of Russian Roulette	Codeforces 1062D	● Nâng cao	Mô phỏng quá trình lật vòng vô hạn và chu trình Euler
19	Physical Collisions of Particles	ICPC Regional Mid	● Nâng cao	Mô phỏng va chạm hạt trên trục tọa độ bằng Timeline Priority Queue
20	Simple Interpreter / LISP Parser	Codeforces 128B / ICPC	● Nâng cao	Mô phỏng trình biên dịch biểu thức và ngăn xếp biến số

● PHẦN 1: CẤP ĐỘ CƠ BẢN (RATING 800 — 1200)

Bài 1: CSES 1068 — Weird Algorithm

- **Mô tả:** Cho số nguyên dương n . Nếu n chẵn, $n = n/2$. Nếu n lẻ, $n = 3n + 1$. Lặp lại đến khi $n = 1$. In ra toàn bộ dãy số.
- **Phân tích & Gợi ý:**
- Dù $n \leq 10^6$ ở input, trong quá trình nhân $3n + 1$, giá trị trung gian có thể vượt quá giới hạn 2×10^9 của kiểu `int`.
- Khởi tạo biến n bằng `long long`.
- **Độ phức tạp:** $O(\text{số bước})$.

Bài 2: CSES 1069 — Repetitions

- **Mô tả:** Cho một chuỗi DNA gồm các ký tự `A`, `C`, `G`, `T`. Tìm độ dài của đoạn con liên tiếp dài nhất chỉ chứa cùng một loại ký tự.
 - **Phân tích & Gợi ý:**
 - Sử dụng một biến `current_len = 1` và `max_len = 1`.
 - Duyệt i từ 1 đến $|S| - 1$: nếu $S[i] == S[i - 1]$ thì `current_len++`, ngược lại `current_len = 1`. Cập nhật `max_len = max(max_len, current_len)`.
 - **Độ phức tạp:** $O(N)$ thời gian, $O(1)$ bộ nhớ phụ.
-

Bài 3: AtCoder ABC 096 C — Grid Repainting 2

- **Mô tả:** Cho bảng kích thước $H \times W$ gồm các ô đen `#` và ô trắng `.`. Ta có thể vẽ được bảng này hay không nếu mỗi nét vẽ chỉ có thể tô màu một cặp 2 ô kề nhau (chung cạnh)?
 - **Phân tích & Gợi ý:**
 - Nhận xét: Bảng hợp lệ khi và chỉ khi **mọi** ô đen `#` đều có ít nhất một ô đen láng giềng kề cạnh với nó.
 - Sử dụng mảng hướng `dx4 = {-1, 0, 1, 0}`, `dy4 = {0, 1, 0, -1}`.
 - Lặp qua từng ô (r, c) : nếu là `#`, kiểm tra xem trong 4 ô láng giềng hợp lệ có ô nào là `#` không. Nếu có một ô `#` đơn độc không có láng giềng đen, in `No`.
 - **Độ phức tạp:** $O(H \times W)$.
-

Bài 4: LeetCode 289 — Conway's Game of Life

- **Mô tả:** Bảng tế bào $M \times N$ gồm các ô sống (1) và chết (0). Mỗi bước thời gian:
 - Ô sống có ít hơn 2 ô sống láng giềng thì chết (do thiếu dân số).
 - Ô sống có 2 hoặc 3 ô sống láng giềng thì tiếp tục sống.
 - Ô sống có nhiều hơn 3 ô sống láng giềng thì chết (do quá đông).
 - Ô chết có đúng 3 ô sống láng giềng thì trở thành ô sống.
 - **Phân tích & Gợi ý:**
 - Các ô cập nhật đồng thời. Không được cập nhật đè lên bảng cũ.
 - Tạo bảng `nextGrid = grid`. Dùng mảng 8 hướng `dx8`, `dy8` để đếm số láng giềng sống quanh ô (r, c) . Áp dụng đúng 4 quy tắc để gán vào `nextGrid[r][c]`.
 - **Độ phức tạp:** $O(M \times N)$.
-

● PHẦN 2: CẤP ĐỘ TRUNG CẤP (RATING 1300 — 1600)

Bài 5: AtCoder ABC 167 D — Teleportation

- **Mô tả:** Có N thị trấn đánh số từ 1 đến N . Từ thị trấn i , một cổng dịch chuyển sẽ đưa bạn đến A_i . Bạn xuất phát từ thị trấn 1 và thực hiện dịch chuyển đúng K lần ($K \leq 10^{18}$). Hỏi bạn đang ở thị trấn nào?
- **Phân tích & Gợi ý:**
- Vì số thị trấn $N \leq 2 \times 10^5$, mà số bước $K \leq 10^{18}$ rất lớn.

- Theo nguyên lý Dirichlet, trong vòng N bước đầu tiên, chắc chắn bạn sẽ ghé thăm lại một thị trấn đã từng đi qua, tạo thành chu trình tuần hoàn.
- Lưu mảng `visited_step[u]` ghi lại bước đầu tiên đến u .
- Khi gặp lại v tại bước `cur_step`: độ dài chu kỳ $L = \text{cur_step} - \text{visited_step}[v]$.
- Số bước còn lại là $(K - \text{cur_step}) \pmod L$. Lăn theo chu kỳ để tìm đáp án.
- **Độ phức tạp:** $O(N)$ thời gian và bộ nhớ.

Bài 6: Codeforces 1154D — Walking Robot

- **Mô tả:** Robot có pin (dung lượng tối đa b , hiện có b) và ắc quy tích lũy năng lượng mặt trời (dung lượng tối đa a , hiện có a). Robot đi trên đoạn đường n đoạn. Mỗi đoạn i có thể có nắng ($s_i = 1$) hoặc râm ($s_i = 0$). Mỗi bước đi tốn 1 đơn vị năng lượng (từ pin hoặc ắc quy). Nếu đi vào đoạn có nắng bằng pin, ắc quy được sạc thêm 1 đơn vị (tối đa a). Hỏi robot đi xa nhất được bao nhiêu đoạn?
- **Phân tích & Gợi ý:**
- Chiến lược mô phỏng tham lam:
 - Nếu đoạn i có nắng ($s_i = 1$): Ta nên ưu tiên dùng pin nếu ắc quy chưa đầy ($a < \text{max_a}$) và pin còn ($b > 0$), vì khi đó ta vừa đi được vừa hồi phục 1 điểm cho ắc quy! Nếu ắc quy đã đầy hoặc hết pin, ta buộc phải dùng ắc quy.
 - Nếu đoạn i không có nắng ($s_i = 0$): Ta ưu tiên dùng ắc quy (để dành pin cho các đoạn có nắng tiếp theo). Nếu hết ắc quy mới dùng đến pin.
- Dừng lại khi cả 2 nguồn năng lượng đều cạn kiệt ($a = 0$ và $b = 0$).
- **Độ phức tạp:** $O(N)$.

Bài 7: CSES 1164 — Room Allocation

- **Mô tả:** Có n khách hàng đến khách sạn. Khách thứ i đến vào ngày a_i và rời đi vào ngày b_i . Mỗi phòng chỉ chứa được 1 khách tại một thời điểm. Cần ít nhất bao nhiêu phòng và phân bổ cụ thể từng khách vào phòng nào?
- **Phân tích & Gợi ý:**
- Đây là bài toán mô phỏng hướng sự kiện (Timeline Simulation).
- Sắp xếp các khoảng thời gian theo ngày đến a_i tăng dần.
- Sử dụng một Min-Priority Queue lưu các phòng đang có người ở: mỗi phần tử là cặp `(departure_day, room_id)`.
- Với mỗi khách mới đến vào ngày a_i :
 - Kiểm tra xem phòng có ngày rời đi sớm nhất trong PQ đã trống chưa (`top.departure_day < a_i`).
 - Nếu đã trống: Tái sử dụng phòng này cho khách mới, cập nhật lại ngày rời đi của phòng và đẩy lại vào PQ.
 - Nếu chưa có phòng nào trống: Cấp thêm 1 phòng mới (tăng số lượng phòng lên 1) và đẩy vào PQ.
- **Độ phức tạp:** $O(N \log N)$.

Bài 8: Design Snake Game

- **Mô tả:** Thiết kế trò chơi Rắn săn mồi trên màn hình kích thước $W \times H$. Rắn di chuyển theo 4 hướng U, D, L, R. Khi ăn thức ăn, chiều dài tăng thêm 1. Trò chơi kết thúc nếu đầu rắn đâm vào tường hoặc tự đâm vào thân mình.
- **Phân tích & Gợi ý:**
- Dùng `std::deque<pair<int, int>>` `body` lưu các đoạn thân rắn (đầu ở `front`, đuôi ở `back`).
- Dùng `std::set<pair<int, int>>` `bodySet` để kiểm tra va chạm thân trong $O(1)$ hoặc $O(\log N)$.
- **Cực kỳ lưu ý:** Khi di chuyển 1 bước không ăn mồi, đuôi rắn được rút đi TRƯỚC KHI đầu mới được thêm vào. Do đó, đầu mới có thể di chuyển vào đúng ô mà đuôi cũ vừa rút ra mà không bị tính là tự cắn vào mình!
- **Độ phức tạp:** $O(1)$ cho mỗi bước di chuyển.

● PHẦN 3: CẤP ĐỘ NÂNG CAO (RATING 1700 — 2000)

Bài 9: Game Engine 2048

- **Mô tả:** Mô phỏng bảng game 2048 kích thước 4×4 . Mỗi lệnh gồm 1 trong 4 hướng trượt: LEFT, RIGHT, UP, DOWN. Khi trượt, các ô di chuyển sát về hướng đó. Hai ô có cùng giá trị chạm nhau sẽ gộp lại thành 1 ô có giá trị gấp đôi. Trong một lần trượt, mỗi ô chỉ được gộp tối đa một lần.
- **Phân tích & Gợi ý:**
- Kỹ thuật chuẩn hóa: Chỉ cần viết **duy nhất một hàm** trượt sang trái `slideLeft(row)`.
- Muốn trượt RIGHT: Lật ngược hàng (reverse) $\rightarrow$ gọi `slideLeft` $\rightarrow$ lật ngược lại.
- Muốn trượt UP hoặc DOWN: Chuyển vị ma trận (Transpose) $\rightarrow$ biến cột thành hàng $\rightarrow$ gọi xử lý $\rightarrow$ chuyển vị ngược lại.
- Xử lý `slideLeft` cho 1 hàng gồm 4 phần tử:
 1. Lọc bỏ các số 0.
 2. Duyệt từ trái sang phải: nếu `arr[i] == arr[i+1]` thì gộp `arr[i] *= 2`, xóa `arr[i+1]`.
 3. Đẩy các số về đầu, đệm các số 0 vào đuôi cho đủ kích thước 4.
- **Độ phức tạp:** $O(1)$ cho mỗi thao tác trượt bảng 4×4 .

Bài 10: ICPC Regional — Physical Collisions of Particles on Line

- **Mô tả:** Có N hạt chuyển động trên trục số thực. Hạt i bắt đầu tại vị trí X_i với vận tốc V_i . Khi hai hạt va chạm, chúng triệt tiêu lẫn nhau hoặc đổi hướng theo định luật bảo toàn. Xác định vị trí và thời điểm của các vụ va chạm đầu tiên.
- **Phân tích & Gợi ý:**
- Sự kiện va chạm giữa 2 hạt kề nhau i và $i + 1$ (đã sắp xếp theo vị trí X) xảy ra khi $V_i > V_{i+1}$ tại thời điểm $t = (X_{i+1} - X_i) / (V_i - V_{i+1})$.
- Sử dụng Danh sách liên kết đôi (Doubly Linked List) để duy trì thứ tự các hạt còn sống sót trên trục tọa độ.

- Sử dụng Hàng đợi ưu tiên (Priority Queue) lưu các sự kiện va chạm `(time, left_particle, right_particle)`.
 - Khi một vụ va chạm xảy ra giữa A và B :
 - Nếu một trong hai hạt đã bị triệt tiêu từ trước $\rightarrow$ Bỏ qua sự kiện (Lazy Deletion).
 - Ngược lại, đánh dấu cả hai hạt đã chết, xóa A và B khỏi Doubly Linked List, và tính thời điểm va chạm tiềm năng mới giữa hạt liền trước A và hạt liền sau B , đẩy sự kiện mới vào PQ.
 - **Độ phức tạp:** $O(N \log N)$.
-
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.